

“초등부 3번 / 고등부 1번. 반품 회수” 문제 풀이

작성자: 나정휘

부분문제 3

모든 $1 \leq i \leq N$ 에 대해 $T_i = 0$ 이므로 트럭은 집을 방문하는 시간과 관계없이 항상 물건을 회수할 수 있다. 따라서 정답은 모든 집을 방문한 다음 위치 0으로 돌아오는 시간인 $2 \times X_N$ 이다.

부분문제 4

모든 $2 \leq i \leq N$ 에 대해 $T_{i-1} \leq T_i$ 가 성립하므로 N 번 집이 가장 반품할 물건을 늦게 내놓는다. 이런 상황에서는 먼저 N 번 집의 위치 X_N 으로 이동해 물건을 회수할 수 있을 때까지 기다린 다음, 다시 위치 0으로 돌아가는 길에 $N-1, N-2, \dots, 1$ 번 집에서 내놓은 물건을 모두 회수하면 물건을 회수하는 데 걸리는 시간을 최소화할 수 있다. 정답은 $X_N + \max(X_N, T_N)$ 이다.

부분문제 5

부분문제 4와 같이 (1) N 번 집으로 이동한 다음, (2) M 시간 동안 기다린 뒤, (3) 중간에 멈추지 않고 위치 0으로 이동하는 방법으로 모든 물건을 회수한다. (1)과 (3)의 소요 시간은 각각 X_N 으로 동일하므로 (2)의 소요 시간인 M 을 최소화해야 한다.

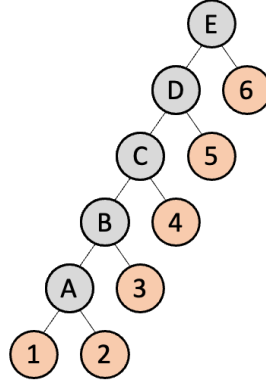
N 번 집의 위치 X_N 에서 위치 0으로 이동할 때 쉬지 않고 이동하면서 모든 물건을 회수하기 위해서는 X_i 를 시각 T_i 또는 그 이후에 지나가야 한다. 이때 (1)에서 X_N , (3)의 과정에서 X_N 에서 X_i 로 이동하는 데 $X_N - X_i$ 만큼의 시간을 소모했으므로, $X_N + (X_N - X_i) + M \geq T_i$ 가 성립해야 하는 것을 알 수 있다. 식을 정리하면 $M \geq T_i - (2X_N - X_i)$ 가 되고, 따라서 $M = \max_{1 \leq i \leq N} \{T_i - (2X_N - X_i)\}$ 이다. 정답은 $2X_N + M$ 이다.

“중등부 3번 / 고등부 2번. 이진 트리” 문제 풀이

작성자: 박찬솔

부분문제 3

T_6 의 형태는 아래와 같다.



노드 A, B, C, D, E가 덮는 리프 노드의 구간은 $[1, 2]$, $[1, 3]$, $[1, 4]$, $[1, 5]$, $[1, 6]$ 이다. 따라서, $f(1, i) = 1$ 이다.

$i \geq 2$ 인 경우, 번호가 i 이상 j 이하인 리프 노드를 덮으려면, 번호가 i 이상 j 이하인 리프 노드들을 모두 선택할 수밖에 없다. 따라서, $f(i, j) = j - i + 1$ 이다.

즉, $S(T_N) = \sum_{j=1}^N 1 + \sum_{i=2}^N \sum_{j=i}^N (j - i + 1)$ 이다. 이 값은 $O(1)$ 에 계산할 수 있다.

부분문제 4

모든 (a, b) 순서쌍에 대해 $f(a, b)$ 의 값을 직접 구하려고 한다.

주어진 트리를 깊이 우선 탐색 등의 방법으로 순회하면, 각 노드 u 를 선택했을 때 덮이는 리프 노드 번호의 구간 $[l_u, r_u]$ 를 계산해 둘 수 있다.

$g(u, a, b)$ 를, “구간 $[a, b]$ 가 노드 u 의 서브 트리에 포함됨을 알고 있을 때, $f(a, b)$ 의 값”이라고 정의하자. $f(a, b)$ 의 값은 $g(\text{root}, a, b)$ 함수의 값을 계산하면 구할 수 있다. g 의 값을 재귀적으로 계산하여, $f(a, b)$ 의 값을 구할 것이다.

- $a = l_u$ 이고 $b = r_u$ 라면, 노드 u 를 선택하면 구간 $[a, b]$ 가 덮이므로, $g(u, a, b) = 1$ 이다. 노드 u 가 리프 노드였다면, 반드시 여기에 해당한다는 점에 유의하라.
- 노드 u 의 왼쪽 자식을 v , 오른쪽 자식을 w 라고 하자. $l_u = l_v$, $r_v + 1 = l_w$, $r_w = r_u$ 이다.
- 구간 $[a, b]$ 가 왼쪽 자식 v 의 서브 트리에 포함된다면 (즉 $b \leq r_v$ 라면), $g(u, a, b) = g(v, a, b)$ 이다.
- 구간 $[a, b]$ 가 오른쪽 자식 w 의 서브 트리에 포함된다면 (즉 $l_w \leq a$ 라면), $g(u, a, b) = g(w, a, b)$ 이다.
- 이제 $a \leq r_v < l_w \leq b$ 이다. 구간 $[a, r_v]$ 는 왼쪽 자식 v 의 서브 트리에 포함되고, 구간 $[l_w, b]$ 는 오른쪽 자식 w 의 서브 트리에 포함되며, 서로 겹치지 않으므로 독립적으로 계산할 수 있다. 즉, $g(u, a, b) = g(v, a, r_v) + g(w, l_w, b)$ 이다.

부분문제 1

위와 같은 g 의 정의는, 완전 이진 트리 형태로 표현된 세그먼트 트리에서, 닫힌 구간 $[a, b]$ 의 정보를 추출하기 위해 세그먼트 트리를 순회하는 함수와 그 형태가 유사하다. $f(a, b)$ 의 값은 세그먼트 트리에서 구간 $[a, b]$ 에 대한 정보를 계산하기 위해, 값을 가져와야 하는 노드의 최소 개수라고 생각할 수 있다.

해당 함수를 사용하여 $O(\log |T_N|)$ 시간에 $f(a, b)$ 를 구할 수 있다. 모든 (a, b) 순서쌍에 대해 $f(a, b)$ 의 값을 구한 후 합쳐주면 된다.

부분문제 5

부분문제 4의 풀이는 모든 순서쌍 (a, b) 에 대해, 구간 $[a, b]$ 를 덮기 위해 최소한으로 선택해야 하는 노드들을 직접 구한다고 볼 수 있다.

관점을 바꿔서, 각각의 노드에 대해, 위의 절차에서 그 노드를 선택하는 순서쌍 (a, b) 의 개수가 몇 개인지를 세어보자.

부분 문제 4에서와 같이, 트리 전체를 순회하여, 각 노드 u 를 선택했을 때 덮이는 리프 노드 번호의 구간 $[l_u, r_u]$ 을 계산해 두었다고 하자.

노드 u 의 부모 노드가 p 일 때, 구간 $[a, b]$ 를 덮을 때 노드 u 가 선택되는 조건은, 아래 두 조건과 같다.

- 노드 u 를 선택했을 때 구간 $[a, b]$ 바깥의 리프 노드가 덮이면 안 된다. 즉, $a \leq l_u \leq r_u \leq b$ 여야 한다.
- 노드 p 를 선택했을 때, 반드시 구간 $[a, b]$ 바깥의 리프 노드 하나 이상은 덮여야 한다. 만약 그렇지 않다면, u 대신 p 를 선택하면 되기 때문이다. 즉, $l_p < a$ 이거나 $b < r_v$ 여야 한다.

위의 두 조건을 동시에 만족하는 (a, b) 의 범위는 직사각형 형태로 나타나며, 그 개수를 $O(1)$ 시간에 구할 수 있다.

모든 노드 u 에 대해 위와 같이 (a, b) 의 개수를 세어 준 후 합쳐 주면, $S(T)$ 를 $O(T$ 의 노드 개수) 시간에 계산할 수 있다.

부분문제 6

부분문제 2와 같이 완전 이진 트리가 주어지는 경우 리프 노드 개수가 2^N 개로 상당히 많으므로, 모든 노드를 순회하는 방식으로 계산할 수 없다.

T 의 루트 노드가 u , u 의 왼쪽 자식 노드가 v , 오른쪽 자식 노드가 w 라고 하자. $f(a, b)$ 의 값을 제시된 대로 부분문제 4에서 제시된 것과 같이 여러 경우를 나누어서 계산해 보자.

$$\begin{aligned} S(T_u) &= \sum_{l_u \leq a \leq b \leq r_u} f(a, b) \\ &= \sum_{l_v \leq a \leq b \leq r_v} f(a, b) + \sum_{l_w \leq a \leq b \leq r_w} f(a, b) + \sum_{l_v \leq a \leq r_v < l_w \leq b \leq r_w} \{f(a, r_v) + f(l_w, b)\} - 1 \end{aligned}$$

마지막에 1을 빼 주는 이유는, $a = l_u, b = r_u$ 일 때 $f(a, b) = 1$ 인 케이스를 간단하게 처리해 주기 위함이다.

$\sum_{l_v \leq a \leq b \leq r_v} f(a, b)$ 는 v 의 서브 트리 내의 모든 (a, b) 순서쌍에 대한 $f(a, b)$ 의 합이므로, $S(T_v)$ 와 같다.

$\sum_{l_w \leq a \leq b \leq r_w} f(a, b)$ 는 w 의 서브 트리 내의 모든 (a, b) 순서쌍에 대한 $f(a, b)$ 의 합이므로, $S(T_w)$ 와 같다.

이제 $\sum_{l_v \leq a \leq r_v < l_w \leq b \leq r_w} \{f(a, r_v) + f(l_w, b)\}$ 의 값을 계산하자. $f(a, r_v)$ 에는 b 가 나타나지 않고, $f(l_w, b)$ 에는 a 가 나타나지 않는다는 점에서 착안하여, 이 식을 전개하면 아래와 같다.

$$\begin{aligned} & \sum_{l_v \leq a \leq r_v < l_w \leq b \leq r_w} \{f(a, r_v) + f(l_w, b)\} \\ &= \sum_{l_v \leq a \leq r_v < l_w \leq b \leq r_w} f(a, r_v) + \sum_{l_v \leq a \leq r_v < l_w \leq b \leq r_w} f(l_w, b) \\ &= (r_w - l_w + 1) \sum_{l_v \leq a \leq r_v} f(a, r_v) + (r_v - l_v + 1) \sum_{l_w \leq b \leq r_w} f(l_w, b) \end{aligned}$$

$r_w - l_w + 1$ 는 w 의 서브 트리 내의 리프 노드 개수이다. $r_v - l_v + 1$ 는 v 의 서브 트리 내의 리프 노드 개수이다. 노드 u 의 서브 트리 내의 리프 노드 개수를 $\text{cnt}[u]$ 라고 정의하자.

$\sum_{l_v \leq a \leq r_v} f(a, r_v)$ 는 v 의 서브 트리에 있는 모든 리프 노드 a 에 대해, $f(a, r_v)$ 의 합이다. 이 값을 $\text{suffix}[v]$ 라고 정의하자.

$\sum_{l_w \leq b \leq r_w} f(l_w, b)$ 는 w 의 서브 트리에 있는 모든 리프 노드 b 에 대해, $f(l_w, b)$ 의 합이다. 이 값을 $\text{prefix}[w]$ 라고 정의하자.

이 값들을 관리할 수 있다면, $S(T_i) = S(T_{A_i}) + S(T_{B_i}) + \text{suffix}[A_i] \times \text{cnt}[B_i] + \text{prefix}[B_i] \times \text{cnt}[A_i] - 1$ 으로 계산할 수 있다.

$\text{cnt}[0] = 1$ 이고, $\text{cnt}[i] = \text{cnt}[A_i] + \text{cnt}[B_i]$ 이다.

$\text{prefix}[i] = \text{prefix}[A_i] + (\text{prefix}[B_i] + \text{cnt}[B_i] - 1)$ 이다. 왼쪽 서브 트리의 f 값들은 그대로 유지되고, 오른쪽 서브 트리의 f 값들은 1 증가하며, 루트 노드의 f 값은 1이기 때문이다.

$\text{suffix}[i] = (\text{suffix}[A_i] + \text{cnt}[A_i] - 1) + \text{suffix}[B_i]$ 이다. 오른쪽 서브 트리의 f 값들은 그대로 유지되고, 왼쪽 서브 트리의 f 값들은 1 증가하며, 루트 노드의 f 값은 1이기 때문이다.

따라서 필요한 모든 값들을 상수 시간에 계산할 수 있으며, 문제를 $O(N)$ 에 해결할 수 있다.

“고등부 3번. 오름차순” 문제 풀이

작성자: 이종영

부분문제 1

D_i 를 $A_i \leq A_{i+1}2^{D_i}$ 를 만족하는 최대의 정수라고 정의하자. 이 배열은 $\log$ 함수를 이용해 $O(N)$ 에 구하거나, 직접 반복문을 이용해 $O(N \log A)$ 에 구할 수 있다. $i = l$ 부터 $r - 1$ 까지 순회하며, $D'_j = D_j$ 로 초기화된 배열에 대해, $D'_i > 0$ 이라면 답에 D'_i 를 더하고, D'_{i+1} 에 D'_i 를 더하는 방법으로 시간 복잡도 $O(NQ)$ 에 문제를 해결할 수 있다. 자세한 내용은 "두 배" 문제의 풀이를 참고하면 좋다.

부분문제 2

$[l, r]$ 에서의 답과 $[l, r + 1]$ 에서의 답을 구하는 과정은 마지막 D'_r 에 대한 처리를 제외하면 동일하다. 각 l 에 대해 r 을 증가시키면서 모든 답을 전처리하면 시간 복잡도 $O(N^2 + Q)$ 에 문제를 해결할 수 있다.

부분문제 4

부분문제 1의 풀이에서 $D'_i > 0$ 대신 $D'_i \geq 0$ 을 기준으로 해도 변함이 없음을 알 수 있다. 또, $A_i \geq A_{i+1}$ 이라는 조건 안에서는 $D_i \geq 0$ 을 만족하므로 $D'_i \geq 0$ 을 만족한다. 즉, 답은 $(r - l)D_l + \dots + D_{r-1}$ 이다. $(r - l)D_l + \dots + D_{r-1} = r(D_l + \dots + D_{r-1}) - (lD_l + \dots + (r - 1)D_{r-1})$ 을 만족하므로 D_i 의 부분합 및 iD_i 의 부분합 배열을 구해둔 후 $O(1)$ 에 쿼리를 답할 수 있다.

부분문제 5

구간 내의 첫 2를 기준으로 1과 2로 최종 값이 정해진다. 구간 내의 첫 2는 동적 프로그래밍을 통해 각 i 에 대해 인덱스가 i 이상인 가장 앞의 2의 위치를 구해둔다면 $O(1)$ 에 구할 수 있다. 실제 비용은 1의 개수를 저장하는 부분합 배열을 이용하면 $O(1)$ 에 구할 수 있다.

부분문제 6

부분문제 5의 풀이를 응용해, 최종 값이 2^k 가 되는 구간을 각 k 에 대해 구하는 방식으로 해결할 수 있다.

부분문제 7

$next_i$ 를 $i + 1$ 부터 순회했을 때, $D'_j < 0$ 이 되는 최소 j 라고 정의하자. D_i 의 부분합 배열 S 에 대해, $next_i$ 는 $S_j < S_i$ 이며 $i < j$ 를 만족하는 최소 j 이다. 이러한 경우 $next$ 배열을 스택을 통해 $O(N)$ 에 구할 수 있음이 잘 알려져 있다. 구체적으로는, i 를 N 부터 1까지 순회한다. 현재 스택의 크기가 1 이상이며 스택의 top j 에 대해 $S_j \geq S_i$ 를 만족한다면 그렇지 않을 때까지 스택에서 pop 연산을 해 준다. 이후 스택의 크기가 1 이상이라면 스택의 top j 가 $next_i$ 가 된다. (스택의 크기가 0이라면 $next_i$ 는 존재하지 않으며, 편의상 이 경우 $next_i = N$ 으로 정의하자.) 마지막으로 스택에 i 를 push하면 된다.

이제 $[l, r]$ 쿼리에 대해 D' 값이 0 이상으로 유지되는 구간들은 $[l, next_{l-1} - 1], [next_{l-1} + 1, next_{next_{l-1} - 1} - 1], \dots$ 이다. $cost_i$ 를 $[i + 1, next_i - 1]$ 구간에 대한 비용으로 정의하자. 부분문제 4의 풀이를 활용하면 $cost$ 배열을 구할 수 있다. 이제 각 쿼리에 대해 $i = l - 1$ 로 시작해, $next_i \leq r - 1$ 인 동안 답에 $cost_i$ 를 더한 후, $i = next_i$ 로 이동한다. 마지막으로 $[i + 1, r - 1]$ 구간에 대한 비용을 더해주면 쿼리에 대한 답을 구할 수 있다.

이 방식은 얼핏 보기에는 $next$ 배열을 타고 가는 것이 최악의 경우 $O(N)$ 이 소요된다고 생각할 수 있지만, $2A_{i+1} \leq A_{next_{i+1}}$ 을 만족하기 때문에 사실은 $O(\log A)$ 의 시간 복잡도를 가진다. 따라서 문제를 $O(N +$

$Q \log A$) 시간 복잡도에 해결할 수 있다. 모범 코드의 경우 D 배열을 $O(N \log A)$ 에 계산해 $O((N + Q) \log A)$ 의 시간 복잡도를 가지며, 이 방식 또한 문제를 해결하는 데에 충분하다.

$next$ 배열을 타고 가는 것이 $O(\log A)$ 가 소요된다는 관찰을 하지 않거나, $next_i$ 를 $S_j \leq S_i$ 이며 $i < j$ 를 만족하는 최소 j 로 정의한 경우에도 문제를 해결할 수 있다. 이 경우 sparse table을 활용한 $O((N + Q) \log N)$ 방식, jump table을 활용한 $O(N + Q \log N)$ 방식 등이 가능하다.

두 방식을 합치면 거의 $O(N + Q)$ 와 유사한 $O((N + Q) \log \log A)$, $O(N + Q \log \log A)$ 의 시간 복잡도로도 문제를 해결할 수 있다.